

Software Architecture and Design Specification

Project: Web Based ML Model Evaluator

Version: 1.0

Authors: Team 16 – Parigna P V, Reia S, Divakar K S, Nikhil K R

Date: 04-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 04-10-2026 | Team 16 | Initial draft of the architecture and design specification.

Approvals

Role	Name	Signature/Date
QA Lead	Parigna P V	Parigna P V- 4/9/26
Test Engineer	Divakar K S	Divakar K S- 4/9/26
Developer	Nikhil K R	Nikhil K R- 4/9/26
Product owner	Reia S	Reia S -4/9/26

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Web Based ML Model Evaluator, a web application that lets users upload datasets, preprocess them, train machine learning models, evaluate them with standard metrics, and compare the results across experiments.

1.2 Scope

Covers the user-facing services of the evaluator: login, dataset upload and validation, preprocessing and train/test split, model selection and training, evaluation, experiment history and comparison, and report generation.

Out of scope: production deployment of trained models, distributed training, and automated hyperparameter search.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

ML (Machine Learning), SAD (Software Architecture Document), SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), ADR (Architecture Decision Record), REST (Representational State Transfer), API (Application Programming Interface), TLS (Transport Layer Security), RBAC (Role-Based Access Control).

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, ADRs, threat models, and API design.

2.2 Related Documents

SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix).

3. Architecture

3.1 Goals & Constraints

Goals: accurate and reproducible model evaluation, secure handling of user data, a simple end-to-end workflow, reliable execution of training jobs, and easy addition of new models.

Constraints: delivered as a single web application by Team 16, built on standard Python ML libraries, limited dataset size and training time (controlled sample datasets), and all communication over HTTPS.

3.2 Stakeholders & Concerns

Users (students, analysts): ease of use, clear and correct results, reasonable response time

Administrators: access control, monitoring, maintainability

Instructors / evaluators: traceability to requirements, correctness of metrics, documentation quality

Developers and QA: modularity, extensibility of models and metrics, testability.

Security auditors: protection of uploaded datasets and user credentials.

3.3 Component (UML) Diagram

UML Component Diagram: User → Web Frontend → Backend API → Authentication, Dataset, Model, Evaluation and Experiment Services → ML Engine / Database.

3.4 Component Descriptions

- Web Frontend: user screens, dataset selection and result display.
- Authentication Service: login and access control.
- Dataset Service: upload, validation, profiling and preprocessing.
- Model Service: supported models and parameters.
- ML Engine: training, prediction and evaluation.
- Experiment Service: history, comparison and reports.
- Database: users, experiments, results and logs.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen because the project is small enough for a single application while still having clear responsibilities. Microservices were not selected because they would add unnecessary deployment and communication complexity.

3.6 Technology Stack & Data Stores

Frontend: JavaScript/React or equivalent. Backend: Python/Django or JavaScript/MERN as selected by Team 16. ML: Python with scikit-learn or equivalent. Data processing: pandas/NumPy or equivalent. Database: relational or document database selected by the team. Communication: REST/JSON and HTTPS.

3.7 Risks & Mitigations

Risk: large datasets or long training → Mitigation: controlled sample datasets for normal tests and status/background handling for longer jobs.

Risk: library/version mismatch → Mitigation: document and pin dependency versions.

Risk: invalid uploads → Mitigation: validate file type, size and contents before processing.

3.8 Traceability to Requirements

ML-F-001 (Login) → Authentication Service

ML-F-002/F-003 (Upload/validation) → Dataset Service

ML-F-010 (Training) → Model Service / ML Engine

ML-F-012/F-013 (Evaluation) → Evaluation Service

ML-F-015/F-016/F-017/F-018 → Experiment Service.

3.9 Security Architecture

STRIDE: Spoofing → authentication; Tampering → server-side validation; Information Disclosure → HTTPS and access control; Denial of Service → input validation/rate limiting; Elevation of Privilege → role-based authorization.

4. Design

4.1 Design Overview

The design separates the presentation, API, service, ML processing and data layers. This makes the workflow easier to test and allows new models to be added without changing the whole application.

4.2 UML Sequence Diagrams

The two main flows of the system are dataset upload/preprocessing and model training/evaluation. They are shown as sequence diagrams in Figures 2 and 3 (Appendix).

Sequence Diagram 1 – Dataset Upload: User → Frontend → Dataset API → Dataset Service → Validation/Preprocessing → Database → Frontend.

Sequence Diagram 2 – Training/Evaluation: User → Frontend → Training API → Model Service → ML Engine → Evaluation → Experiment Service → Database → Frontend.

4.3 API Design

Interface definitions for the Dataset Service and the Model/Evaluation services. All endpoints require an authenticated user (token in the Authorization header) and use JSON over HTTPS unless noted.

Dataset Service

Endpoint: /api/datasets

Method: POST (multipart/form-data)

Request: {file (CSV), name, targetColumn}

Response: {datasetId, rows, columns, columnTypes, missingValues}

Errors: 400 Invalid or empty file, 401 Unauthorized, 413 File too large, 415 Unsupported file type.

Model / ML Engine Service

Endpoint: /api/experiments

Method: POST

Request: {datasetId, preprocessing: {missingValues, scaling, encoding, testSize}, model, parameters}

Response: {experimentId, status}

Errors: 400 Invalid parameters, 401 Unauthorized, 404 Dataset not found, 422 Model not compatible with dataset.

Evaluation / Experiment Service

Endpoint: /api/experiments/{experimentId}/results

Method: GET

Request: experimentId (path parameter)

Response: {status, metrics: {accuracy, precision, recall, f1, confusionMatrix}, trainingTime}

Errors: 401 Unauthorized, 404 Experiment not found, 409 Training still in progress.

4.4 Error Handling, Logging & Monitoring

Use clear error messages and suitable HTTP status codes. Do not write passwords or secrets to logs. Logs should include timestamps, actions and errors. Monitor API failures, training failures, response time and application availability.

4.5 UX Design

The UI should have clear navigation and readable text. Main flow: Dataset → Preprocessing → Model Selection → Training → Evaluation → Comparison → Report. Progress and validation messages should be shown.

4.6 Open Issues & Next Steps

Possible future enhancements include more ML algorithms, hyperparameter tuning, cross-validation, automated model recommendation, explainable AI features and cloud deployment.

5. Appendices

5.1 Glossary: ML, SAD, SRS, STP, RTM, ADR, REST, API, TLS, RBAC, CSV, STRIDE.

5.2 References: IEEE 42010, OWASP, NIST SP 800-160.

5.3 Tools: PlantUML, draw.io, Swagger.

SRS Requirement	Architecture / Design Component	STP Test
ML-F-001 Login	Authentication Service	TC-AUTH-01
ML-F-002/F-003 Upload & validation	Dataset Service	TC-DATA-01 / TC-DATA-02
ML-F-006/F-007 Preprocessing & split	Preprocessing Service	TC-PREP-02 / TC-PREP-03
ML-F-008/F-009/F-010 Model & training	Model Service + ML Engine	TC-MODEL-01 / TC-TRAIN-01

ML-F-012/F-013 Evaluation	Evaluation Service	TC-EVAL-01 / TC-EVAL-02
ML-F-015/F-016/F-017 Comparison/history	Experiment Service + Database	TC-COMP-01 / TC-HIST-01
ML-F-018 Report	Report Service	TC-REP-01
ML-SR-002/SR-003 Security	Authentication + Security Layer	TC-SEC-02 / TC-SEC-03

UML Diagrams

UML Component Diagram — Web Based ML Model Evaluator

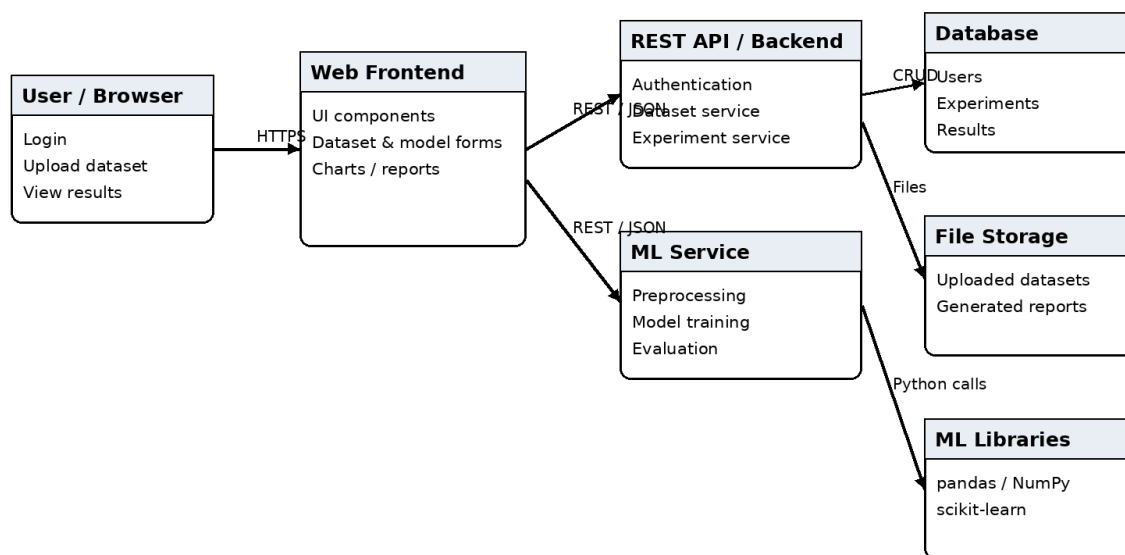


Figure 1. UML Component Diagram

The component diagram shows the major system components and their interfaces.

Sequence Diagram 1: Dataset Upload and Preprocessing

UML Sequence Diagram 1 — Dataset Upload and Preprocessing

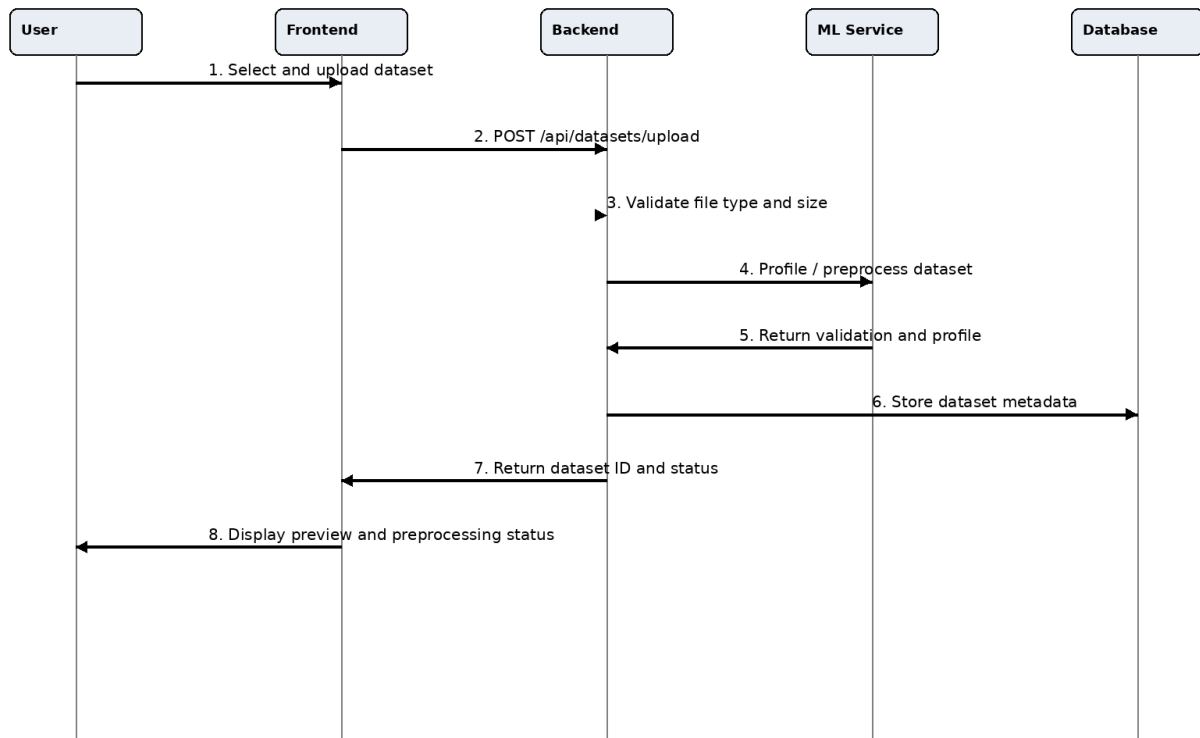


Figure 2. UML Sequence Diagram — Dataset Upload and Preprocessing

Sequence Diagram 2: Model Training and Evaluation

UML Sequence Diagram 2 — Model Training and Evaluation

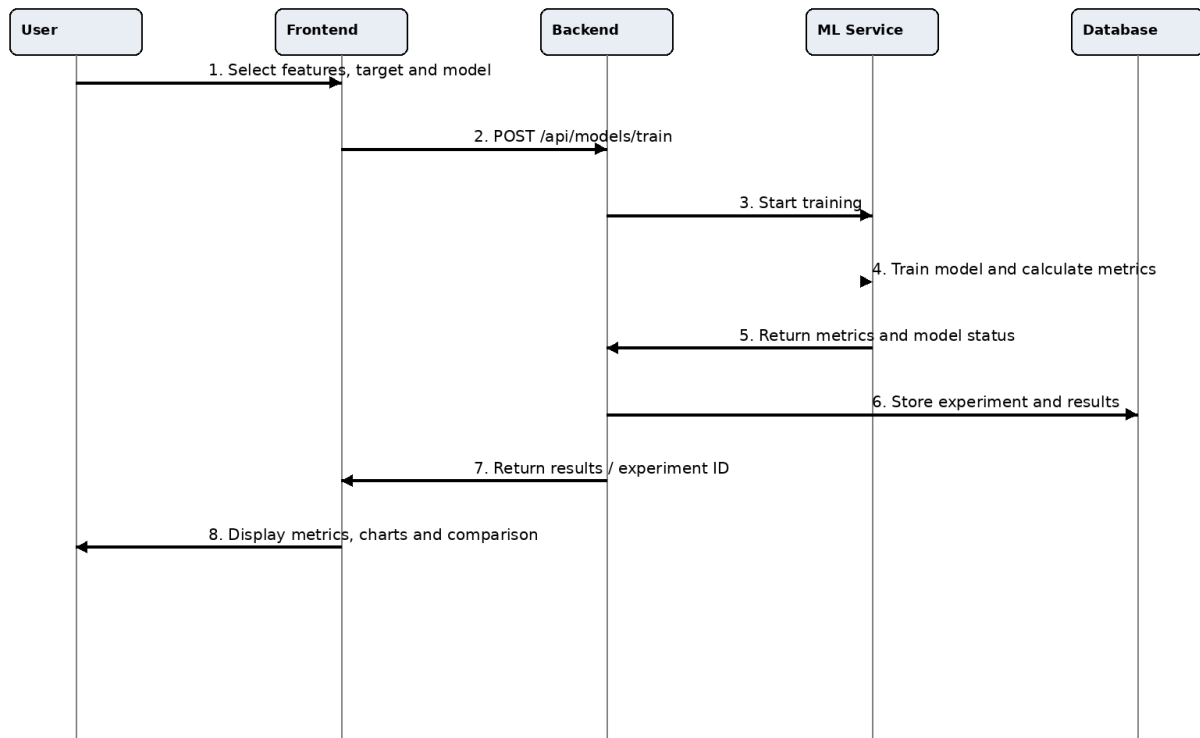


Figure 3. UML Sequence Diagram — Model Training and Evaluation